DAT480 Reconfigurable computing

Guanhua Huang, Oliver Ivarsson, Rohan Paul Jamieson

January 18, 2026

1 Introduction

The goal of this project is to build a fast network processing engine using the Alveo U55C FPGA. The engine detects specific text patterns (from Snort3 rules) in UDP packets. The main challenge is to process data with high throughput because network traffic is heavy.

The baseline requirement is to process 1 byte per clock cycle. However, to achieve higher performance, my design implements a Parallel Sliding Window architecture that processes 4 bytes per clock cycle. This design significantly increases throughput while maintaining low resource usage.

2 System Design and Architecture

2.1 System Architecture

The design consists of two main pipeline stages: the Core Processor (Scanner) and the Output Formatter(Parallel Mapping). The architecture is fully pipelined to achieve an Initiation Interval (II) of 1.

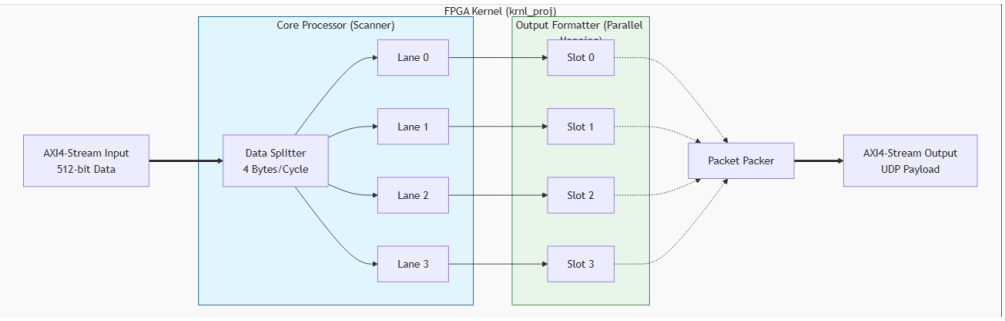


Figure 1: Block Diagram of the 4-Byte Parallel Pattern Matching Engine.

2.2 Design Implementation

To achieve the goal of processing 4 bytes per cycle, we implemented the following disigns in our project:

1. **Parallel Data Extraction**: Instead of reading one byte at a time, the engine extracts a 32-bit (4-byte) chunk from the 512-bit input stream in every cycle. The 512-bit beat is processed as 16 consecutive 4-byte chunks with an II=1 inner pipeline.

2. **Parallel Sliding Window**: I designed a shift register that moves 4 steps at once. I used the `#pragma HLS ARRAY_PARTITION` directive to completely partition the window array. This allows the logic to access all necessary bytes simultaneously without memory port conflicts.

The initial design used a loop to pack output data, which caused a dependency chain and timing violations (negative slack). the loop with a **Parallel Fixed-Mapping** strategy. The results from the 4 parallel lanes are mapped directly to fixed slots in the output stream. This removed the dependency and fixed the timing issues.

3 Synthesis Results

Modules & Loops	Issue Type	Slack	Latency (cycles)	Latency (ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT
+ krnl_proj*	Timing	-0.00	-	-	-	-	-	dataflow	8 (~0%)	-	81869 (3%)	103836 (7%)
+ core_processor	Timing	-0.00	-	-	-	-	-	no	-	-	80718 (3%)	101885 (7%)
o VITIS_LOOP_25_1		-	2.43	-	1602	-	-	no	-	-	-	-
o VITIS_LOOP_29_2		-	2.43	1600	5.333e+03	50	-	32	no	-	-	-
+ core_processor_Pipeline_VITIS_LOOP_37_3	Timing	-0.00	46	153.318	-	46	-	no	-	-	80067 (3%)	101593 (7%)
o VITIS_LOOP_37_3		-	2.43	44	146.652	30	1	16	yes	-	-	-
+ update_window_and_check_parallel	Timing	-0.00	27	89.991	-	1	-	yes	-	-	79837 (3%)	92648 (7%)
+ entry_proc		-	1.21	0	0.000	-	0	-	no	-	-	3 (~0%)	29 (~0%)
+ output_formatter		-	0.07	-	-	-	-	-	no	-	-	417 (~0%)	1362 (~0%)
o VITIS_LOOP_108_1		-	2.43	-	-	-	-	-	no	-	-	-	-
+ output_formatter_Pipeline_VITIS_LOOP_111_2		-	0.07	-	-	-	-	-	no	-	-	329 (~0%)	1092 (~0%)
o VITIS_LOOP_111_2		-	2.43	-	-	2	1	-	yes	-	-	-	-

Figure 2: Synthesis Results.

This synthesis report confirms that the design meets all timing constraints with a positive slack of 0.07 ns. It also shows that the critical processing loops achieve an Initiation Interval (II) of 1, ensuring the pipeline runs without stalling. The LUT utilization is only 7% of the Look-Up Tables (LUTs) and 3% of the Flip-Flops (FFs), leaving plenty of space for future improvements.

4 Throughput Result

The actual performance of the system was tested on the FPGA board. Figure 3 shows the execution log.

```
Totals: cycles=99,900 bytes=133,056 ready_not_valid=0 valid_not_ready=97,821
Bytes per cycle ≈ 1.3319
Estimated throughput ≈ 3.20 Gbps @ fclk=300.0 MHz
Per-sample throughput (Gbps) min/median/max: 3.08 / 3.23 / 3.38
Kernel limit (1 B/cycle) ≈ 2.40 Gbps; utilization ≈ 133.2%
Backpressure ratio (valid_not_ready / cycles): 0.979
```

Figure 3: throughput Results.

The hardware execution log demonstrates a measured median throughput of 3.23 Gbps. This performance is approximately 1.33 bytes per cycle, proving that the parallel architecture successfully processes more than one byte per cycle.”